# Gossipsub 메시지 전파 메커니즘 분석

## 개요

Gossipsub은 libp2p의 pub/sub 라우팅 프로토콜로, meshsub과 randomsub의 조합을 통해 효율적인 메시지 전파를 구현합니다. 이 문서는 Rust libp2p의 Gossipsub 구현에서 메시지가 어떻게 전파되는지 상세히 분석합니다.

## 핵심 구성 요소

### 1. 네트워크 토폴로지

Gossipsub은 두 가지 주요 네트워크 구조를 사용합니다:

**Mesh Network (메시 네트워크)**

```
// 각 토픽별로 메시 피어들을 관리
mesh: HashMap<TopicHash, BTreeSet<PeerId>>
```

- **목적**: 안정적인 메시지 전달을 위한 핵심 네트워크
- **특징**: 각 토픽마다 `mesh_n` 개의 피어와 직접 연결
- **관리**: GRAFT/PRUNE 메시지를 통해 동적으로 관리

**Fanout Network (팬아웃 네트워크)**

```
// 구독하지 않은 토픽에 대한 발행용 피어들
fanout: HashMap<TopicHash, BTreeSet<PeerId>>
```

```
fanout_last_pub: HashMap<TopicHash, Instant>
```

- **목적**: 구독하지 않은 토픽에 메시지를 발행할 때 사용
- **특징**: TTL 기반으로 관리되며, 일정 시간 후 자동 제거

### 2. 메시지 캐시 시스템

#### MessageCache 구조

```
pub(crate) struct MessageCache {
msgs: HashMap<MessageId, (RawMessage, HashSet<PeerId>)>,
iwant_counts: HashMap<MessageId, HashMap<PeerId, u32>>,
history: Vec<Vec<CacheEntry>>,
gossip: usize,
}
```

**주요 기능:**

- **메시지 저장**: 최근 메시지들을 히스토리 기반으로 저장
- **중복 추적**: 동일한 메시지를 받은 피어들을 추적
- **가십 지원**: 최근 `gossip` 개의 히스토리 슬롯에서 메시지 ID 제공

## 메시지 전파 과정

### 1. 메시지 발행 (Publishing)

#### 발행 과정

```
pub fn publish(&mut self, topic: impl Into<TopicHash>, data: impl Into<Vec<u8>>)
-> Result<MessageId, PublishError>
```

**단계별 처리:**

1. **데이터 변환**: `data_transform.outbound_transform()` 적용
2. **크기 검증**: `max_transmit_size_for_topic` 확인
3. **메시지 구성**: `build_raw_message()` 호출
4. **메시지 ID 생성**: 압축되지 않은 데이터로 ID 계산
5. **중복 검사**: `duplicate_cache` 에서 중복 확인
6. **수신자 선택**: 메시 또는 팬아웃 피어 선택
7. **메시지 전송**: 선택된 피어들에게 전송

**수신자 선택 로직**

**Flood Publish 모드:**

```
if self.config.flood_publish() {
// 점수 임계값 이상의 모든 피어와 명시적 피어에게 전송
recipient_peers.extend(peers_on_topic.filter(|p| {
self.explicit_peers.contains(*p) ||
!self.peer_score.below_threshold(p, |ts| ts.publish_threshold).0
}));
}
```

**일반 모드:**

- **메시 피어 우선**: 토픽의 메시 피어들에게 전송
- **부족한 경우**: `mesh_n` 에 도달하도록 추가 피어 선택
- **팬아웃 사용**: 메시가 없는 경우 팬아웃 피어 사용

## 2. 메시지 수신 및 전파

### 메시지 처리 파이프라인

```
fn on_connection_handler_event(&mut self, propagation_source: PeerId,
_connection_id: ConnectionId, handler_event: THandlerOutEvent<Self>)
```

**처리 단계:**

1. **메시지 수신**: 연결 핸들러에서 RPC 메시지 수신
2. **중복 검사**: `duplicate_cache` 에서 중복 확인
3. **검증 대기**: 애플리케이션 검증을 위해 대기
4. **전파 결정**: 검증 후 전파 여부 결정
5. **피어 선택**: 전파할 피어들 선택
6. **메시지 전송**: 선택된 피어들에게 전송

### 전파 피어 선택 알고리즘

```
// 메시 피어들 중에서 선택 (원본 발신자 제외)
let mut mesh_peers_to_forward = mesh_peers.clone();
mesh_peers_to_forward.remove(&propagation_source);

// 점수가 낮은 피어 제외
```

```
mesh_peers_to_forward.retain(|peer| {
!self.peer_score.below_threshold(peer, |ts| ts.publish_threshold).0
});
```

## 3. 가십 메커니즘

### IHAVE/IWANT 프로토콜

### IHAVE 메시지 생성:

```
// 하트비트마다 실행
for (topic_hash, peers) in self.mesh.iter() {
let message_ids = self.mcache.get_gossip_message_ids(topic_hash);
if !message_ids.is_empty() {
// 랜덤하게 선택된 피어들에게 IHAVE 전송
let gossip_peers = get_random_peers(&self.connected_peers, topic_hash,
self.config.gossip_lazy(), |peer| {
!peers.contains(peer) && // 메시 피어 제외
!self.explicit_peers.contains(peer) &&
!self.peer_score.below_threshold(peer, |ts| ts.gossip_threshold).0
});
for peer in gossip_peers {
self.send_message(peer, RpcOut::IHave(IHave {
topic_hash: topic_hash.clone(),
message_ids: message_ids.clone(),
```

```
}));
}
}
}
```

**IHAVE 처리:**

```
fn handle_ihave(&mut self, peer_id: &PeerId, ihave_msgs: Vec<(TopicHash, Vec<Mes
// 가십 임계값 확인
if self.peer_score.below_threshold(peer_id, |ts| ts.gossip_threshold).0 {
return; // 점수가 낮은 피어 무시
}
// 플러드 보호
let peer_have = self.count_received_ihave.entry(*peer_id).or_insert(0);
*peer_have += 1;
if *peer_have > self.config.max_ihave_messages() {
return; // 너무 많은 IHAVE 무시
}
// 필요한 메시지 ID 수집
let mut iwant_ids = HashSet::new();
for (topic, ids) in ihave_msgs {
if !self.mesh.contains_key(&topic) { continue; } // 구독하지 않은 토픽 무시
for id in ids.into_iter().filter(|id| {
!self.duplicate_cache.contains(id) && // 이미 받은 메시지 제외
```

```
!self.gossip_promises.contains(id) // 이미 요청한 메시지 제외
}) {
iwant_ids.insert(id);
}
}
// IWANT 메시지 전송
if !iwant_ids.is_empty() {
self.send_message(*peer_id, RpcOut::IWant(IWant {
message_ids: iwant_ids_vec,
}));
}
}
```

**IWANT 처리:**

```
fn handle_iwant(&mut self, peer_id: &PeerId, iwant_msgs: Vec<MessageId>) {
for id in iwant_msgs {
if let Some((msg, count)) = self.mcache.get_with_iwant_counts(&id, peer_id) {
if count <= self.config.gossip_retransimission() {
// IDONTWANT 확인
if let Some(peer) = self.connected_peers.get_mut(peer_id) {
if peer.dont_send.contains_key(&id) { continue; }
}
```

```
// 캐시된 메시지 전송
self.send_message(*peer_id, RpcOut::Forward {
message: msg.clone(),
timeout: Delay::new(self.config.forward_queue_duration()),
});
}
}
}
}
```

## 4. 메시 관리

### 하트비트 기반 메시 유지

```
fn heartbeat(&mut self) {
for (topic_hash, peers) in self.mesh.iter_mut() {
// 1. 점수가 낮은 피어 제거
let mut to_remove_peers = Vec::new();
for peer_id in peers.iter() {
let peer_score = scores.get(peer_id).map(|r| r.score).unwrap_or_default();
if peer_score < 0.0 {
to_prune.entry(*peer_id).or_insert_with(Vec::new).push(topic_hash.clone());
to_remove_peers.push(*peer_id);
}
```

```
}

// 2. 피어 수가 부족한 경우 추가

if peers.len() < mesh_n_low {

let desired_peers = mesh_n - peers.len();

let peer_list = get_random_peers(&self.connected_peers, topic_hash,

desired_peers, |peer| {

!peers.contains(peer) &&

!explicit_peers.contains(peer) &&

scores.get(peer).map(|r| r.score).unwrap_or_default() >= 0.0

});

for peer in &peer_list {

to_graft.entry(*peer).or_insert_with(Vec::new).push(topic_hash.clone());

}

peers.extend(peer_list);

}

// 3. 피어 수가 많은 경우 제거

if peers.len() > mesh_n_high {

let excess_peer_no = peers.len() - mesh_n;

// 점수 기준으로 정렬하여 낮은 점수 피어부터 제거

// ...

}

}
```

```
}
```

### GRAFT/PRUNE 메시지 처리

#### GRAFT 처리:

```
fn handle_graft(&mut self, peer_id: &PeerId, topics: Vec<TopicHash>) {
for topic_hash in topics {
if let Some(peers) = self.mesh.get_mut(&topic_hash) {
// 백오프 시간 확인
if let Some(backoff_time) = self.backoffs.get_backoff_time(&topic_hash, peer_id)
if backoff_time > now {
// 백오프 중인 피어는 PRUNE으로 응답
to_prune_topics.insert(topic_hash.clone());
continue;
}
}
// 점수 확인
if scores.get(peer_id).map(|r| r.score).unwrap_or_default() < 0.0 {
to_prune_topics.insert(topic_hash.clone());
continue;
}
// 메시 크기 확인
if peers.len() >= mesh_n_high && !is_outbound {
to_prune_topics.insert(topic_hash.clone());
```

```
continue;
}
// 메시에 추가
peers.insert(*peer_id);
}
}
}
```

## 성능 최적화 기법

### 1. 메시지 중복 제거

#### 중복 캐시

```
duplicate_cache: DuplicateCache<MessageId>
```

- **목적**: 이미 처리한 메시지 추적
- **구현**: LRU 캐시 기반
- **효과**: 네트워크 대역폭 절약

#### 발신자 추적

```
// MessageCache에서 중복 발신자 추적
pub(crate) fn observe_duplicate(&mut self, message_id: &MessageId, source: &Peer
if let Some((message, originating_peers)) = self.msgs.get_mut(message_id) {
```

```
if !message.validated {
originating_peers.insert(*source);
}
}
}
```

## 2. 백프레셔 및 흐름 제어

### 우선순위 큐

```
pub(crate) struct Sender {
priority_sender: async_channel::Sender<RpcOut>, // GRAFT, PRUNE, 구독 메시지
non_priority_sender: async_channel::Sender<RpcOut>, // IHAVE, IWANT, Forward
}
```

### 메시지 타임아웃

```
RpcOut::Forward {
message: msg,
timeout: Delay::new(self.config.forward_queue_duration())
}
```

## 3. 피어 스코어링

### 점수 기반 필터링

```
// 가십 임계값 이하 피어 무시
if self.peer_score.below_threshold(peer_id, |ts| ts.gossip_threshold).0 {
return;
}


// 발행 임계값 이하 피어 제외
if self.peer_score.below_threshold(peer, |ts| ts.publish_threshold).0 {
continue;
}
```

# 보안 메커니즘

## 1. 플러드 보호

### IHAVE 플러드 보호

```
let peer_have = self.count_received_ihave.entry(*peer_id).or_insert(0);
*peer_have += 1;
if *peer_have > self.config.max_ihave_messages() {
return; // 너무 많은 IHAVE 무시
}
```

### IWANT 제한

```
if *iasked + iask > self.config.max_ihave_length() {
iask = self.config.max_ihave_length().saturating_sub(*iasked);
}
```

## 2. 백오프 메커니즘

### PRUNE 백오프

```
fn make_prune(&mut self, topic_hash: &TopicHash, peer: &PeerId,
do_px: bool, on_unsubscribe: bool) -> Prune {
let backoff = if on_unsubscribe {
self.config.unsubscribe_backoff()
} else {
self.config.prune_backoff()
};
self.backoffs.update_backoff(topic_hash, peer, backoff);
// ...
}
```

## 3. IDONTWANT 메커니즘

### 불필요한 메시지 차단

```
// 발행 시 IDONTWANT 전송
if raw_message.raw_protobuf_len() > self.config.idontwant_message_size_threshold
```

```
&& self.config.idontwant_on_publish() {
self.send_message(*peer_id, RpcOut::IDontWant(IDontWant {
message_ids: vec![msg_id.clone()],
}));
}


// IWANT 처리 시 IDONTWANT 확인
if let Some(peer) = self.connected_peers.get_mut(peer_id) {
if peer.dont_send.contains_key(&id) {
continue; // 전송하지 않음
}
}
```

## 구성 매개변수

### 메시 관리 매개변수

- `mesh_n` : 메시의 목표 피어 수 (기본값: 6)
- `mesh_n_low` : 메시 피어 수 하한 (기본값: 5)
- `mesh_n_high` : 메시 피어 수 상한 (기본값: 12)
- `mesh_outbound_min` : 최소 아웃바운드 피어 수 (기본값: 2)

### 가십 매개변수

- `gossip_lazy` : 가십할 피어 수 (기본값: 6)

- `gossip_retransimission` : 재전송 허용 횟수 (기본값: 3)
- `history_gossip` : 가십용 히스토리 슬롯 수 (기본값: 3)
- `history_length` : 전체 히스토리 길이 (기본값: 5)

### 타이밍 매개변수

- `heartbeat_interval` : 하트비트 간격 (기본값: 1초)
- `fanout_ttl` : 팬아웃 TTL (기본값: 60초)
- `iwant_followup_time` : IWANT 응답 대기 시간 (기본값: 3초)

## 결론

Gossipsub의 메시지 전파 메커니즘은 다음과 같은 핵심 특징을 가집니다:

1. **효율성**: 메시 네트워크를 통한 안정적 전달과 가십을 통한 효율적 전파
2. **확장성**: 피어 스코어링과 백오프를 통한 대규모 네트워크 지원
3. **보안성**: 플러드 보호와 IDONTWANT를 통한 공격 방어
4. **적응성**: 하트비트 기반 동적 메시 관리

이러한 메커니즘들이 조합되어 안정적이고 효율적인 P2P 메시지 전파 시스템을 구현합니다.